

Contents

- guide 01_clipper_prediction_serving: Clipper Low-Latency Online Prediction Serving** 1
 - 0. 为什么 LLM 时代还要读 Clipper 2
 - 1. 论文故事线 2
 - 1.1 训练和推理的地位不对称 2
 - 1.2 论文要满足的三件事 2
 - 2. Clipper 总体架构 3
 - 3. 请求路径按事件拆解 4
 - 4. Model Abstraction Layer 4
 - 4.1 Common prediction interface 4
 - 4.2 Prediction cache 5
 - 4.3 Adaptive batching 5
 - 4.4 AIMD 动态 batch size 6
 - 4.5 Delayed batching 6
 - 4.6 Model container 和 replica scaling 6
 - 5. Model Selection Layer 7
 - 5.1 Selection policy interface 7
 - 5.2 Exp3 single model selection 7
 - 5.3 Exp4 ensemble model selection 8
 - 5.4 Straggler mitigation 8
 - 5.5 Contextualization 9
 - 6. 和 TensorFlow Serving 的对比 9
 - 7. 局限性 9
 - 8. 和本仓库代码怎么连起来 10
 - 8.1 本专题已有生产 serving 面 10
 - 8.2 新增 Clipper toy 10
 - 8.3 测试对应论文 claim 11
 - 9. 和 LLM production serving 的迁移 12
 - 9.1 API layer 12
 - 9.2 Batching and streaming 12
 - 9.3 Cache 12
 - 9.4 Selection policy 13
 - 10. AI agent 学习方式 13
 - 11. 闭卷自测 13
 - 12. 一句话复述模板 13

guide_01_clipper_prediction_serving: Clipper Low-Latency Online Prediction Serving

原论文: Clipper: A Low-Latency Online Prediction Serving System
本地原文 PDF: [learning/production-serving/paper/01_clipper_prediction_serving.pdf](#)
作者: Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, Ion Stoica
机构: UC Berkeley, The University of Chicago
版本: arXiv 1612.03079v2, 2017-02-28

类型: production prediction serving system

0. 为什么 LLM 时代还要读 Clipper

Clipper 不是 LLM serving 论文。它发表于 2017 年, 当时主角还是 scikit-learn, Spark MLlib, Caffe, TensorFlow, HTK 这类传统机器学习框架。但它讲清楚了一个今天仍然核心的问题:

训练出模型只是开始。

真正困难的是把模型变成稳定、低延迟、可替换、可监控、可扩展的在线服务。

LLM production serving 里你会见到很多新词: OpenAI-compatible API, SSE streaming, vLLM, Triton, TensorRT-LLM, prefix cache, metrics, cost routing。Clipper 的价值是提供更基础的系统骨架:

- application API 和 model runtime 要解耦。
- model 可以来自不同 framework。
- serving 层要管理 latency SLO, throughput, accuracy。
- cache 和 batching 是 serving 层的优化, 不应该散落在每个模型里。
- model selection 可以根据反馈在线变化。
- ensemble 提高准确率, 但会引入 straggler 和 tail latency。
- 生产系统要能承认黑盒模型的边界: slow model remains slow。

一句话:

Clipper 把 prediction serving 从" 启动一个模型进程" 提升成" 围绕模型的一层可控系统"。

这正好对应本专题的代码: OpenAI API server, SSE streaming, Prometheus metrics, cost calculator, TRT-LLM build config, Triton model repo。这些都是在训练之后发生的工程。

1. 论文故事线

1.1 训练和推理的地位不对称

论文开头指出: 当时机器学习系统研究大量关注 training, 比如 Spark, Parameter Server, PowerGraph 等。但 inference/prediction serving 被认为比较容易, 常常只被框架提供离线 batch inference 支持。

这个假设在生产里不成立:

- training 可以跑几个小时甚至几天。
- inference 在用户请求路径上, 要毫秒级响应。
- inference 请求量通常远高于 training 样本访问量。
- 模型和框架会持续变化。
- 线上要面对 flash crowd, concept drift, user feedback, tail latency。

Clipper 的出发点是: prediction serving 是一个独立系统问题。

1.2 论文要满足的三件事

Clipper 说 prediction serving system 要同时实现:

low latency
high throughput
improved accuracy and robustness

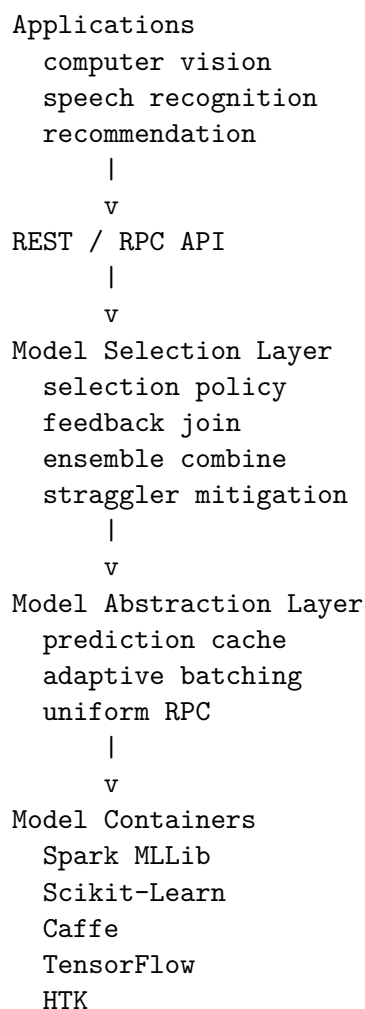
注意第三项。Clipper 不只是做一个快的 RPC wrapper。它把 model selection layer 放在 serving 层, 让系统可以:

- 在多个模型之间选择。
- 组合多个模型的 prediction。
- 用反馈更新选择策略。
- 针对用户、session 或 context 个性化选择状态。
- 在 straggler 出现时仍然在 deadline 内给出结果。

这跟今天的 LLM router 很像: 小模型、强模型、工具模型、thinking model 都可能同时存在, serving 层要决定什么时候调用谁。

2. Clipper 总体架构

论文 Figure 1 的架构可以重画成:



两个核心层:

Model Abstraction Layer:

- 给不同 framework 提供共同 prediction interface。
- 负责 prediction cache。
- 负责 adaptive batching。
- 负责 model container 和 RPC。
- 负责 resource isolation。

Model Selection Layer:

- 决定一个 query 应该发给哪个模型或哪些模型。
- 把多个模型结果组合成 final prediction。
- 通过 feedback 更新模型选择策略。
- 估计 confidence。
- 在 deadline 到达时处理 missing predictions。

这个分层设计的意义是: 应用不需要知道模型在哪个框架里, 模型框架也不需要知道上层业务如何做 selection。

3. 请求路径按事件拆解

一个 query 经过 Clipper 的路径:

1. Application sends prediction request.
2. Model Selection Layer receives request.
3. Selection policy chooses one or more ModelId.
4. Model Abstraction Layer checks prediction cache.
5. Cache hit: return cached prediction.
6. Cache miss: enqueue query into model-specific batching queue.
7. Adaptive batcher forms a mini-batch under latency SLO.
8. Batch is sent via RPC to model container.
9. Model container runs native framework inference.
10. Prediction returns to abstraction layer and cache is populated.
11. Selection layer combines predictions.
12. Final prediction and confidence return to application.
13. Later feedback is joined with cached predictions.
14. Selection policy updates state.

这条路径非常适合今天读 LLM serving:

- OpenAI API request 对应 application API。
- model routing 对应 selection policy。
- prefix cache / response cache 对应 prediction cache。
- dynamic batching 对应 adaptive batching。
- vLLM/Triton/TensorRT-LLM worker 对应 model container。
- Prometheus latency histogram 对应 SLO 和 tail latency monitoring。

4. Model Abstraction Layer

4.1 Common prediction interface

Clipper 把模型封进 container, 只要求实现一个 batch prediction interface。论文 Listing 1 的意思可以写成:

```
Predictor<X, Y>:
  pred_batch(inputs: List<X>) -> List<List<Y>>
```

返回 nested list 是因为一个输入可能产生多个输出。比如 speech recognition 可能输出多个候选 transcription。

Clipper 的重点不是把所有模型转换成一个统一模型格式, 而是用 narrow waist API 让模型各自留在原 framework 中。论文实现支持 Spark MLlib, Scikit-Learn, Caffe, TensorFlow, HTK, 而且多数 framework/container 的接入只需要很少代码。

这对今天的启发:

- 不一定要把所有模型都塞进同一个 engine。
- serving 层可以统一 API, metrics, routing, cache。
- 底层 model runtime 可以是 vLLM, Triton, llama.cpp, Ollama, TensorRT-LLM。

4.2 Prediction cache

Clipper 的 cache 是 function cache:

```
Predict(model_id, x) -> y
```

key 不是只有 query, 而是:

```
(model_id, query)
```

因为不同模型对同一个 query 的 prediction 可能不同。

cache 作用有两个:

1. 热门 query 直接复用 prediction, 降低 latency 和 model load。
2. feedback 到来时, selection layer 需要把 feedback 和之前的 prediction 连接起来, cache 可以保存近期 prediction。

论文给了一个例子: 小 ensemble 下, prediction caching 把 feedback processing throughput 从大约 6K observations/s 提高到 11K observations/s, 约 1.6x。

Clipper 用 LRU/CLOCK eviction。你不需要背 CLOCK 细节, 重点是:

```
cache is serving state,  
not model state.
```

今天 LLM 里的 prefix cache 和 response cache 也要问同样问题:

- key 是什么。
- 哪些模型共享。
- 什么时候失效。
- cache hit 是否改变 correctness。
- cache savings 如何计入成本。

4.3 Adaptive batching

batching 的矛盾:

larger batch:

```
higher throughput  
better GPU or BLAS utilization  
amortized RPC overhead
```

but:

```
first query waits longer  
tail latency may violate SLO
```

Clipper 的设计是明确使用 latency SLO 来自动调 batch size。目标:

```
choose max batch size that maximizes throughput  
subject to P99 batch latency staying under SLO
```

论文 Figure 3 展示不同模型的 latency profile 差异非常大。在 20ms SLO 下, linear SVM 和 kernel SVM 的最大 batch size 差异达到 241x; linear SVM 可以接近 30,000 qps, kernel SVM 只有约 200 qps。

这说明 batch size 不能全局写死。每个 model container 都应该有自己的 batching queue 和 batch profile。

4.4 AIMD 动态 batch size

Clipper 默认用 AIMD:

```
if batch_latency <= SLO:
    max_batch_size = max_batch_size + additive_step

if batch_latency > SLO:
    max_batch_size = floor(max_batch_size * multiplicative_backoff)
```

论文也试过 quantile regression 来预测 P99 latency 和 batch size 的关系, 但最后偏好 AIMD, 因为:

- 实现更简单。
- 调参更容易。
- 对 framework pause 或 throughput 变化更鲁棒。

Figure 4 结果: adaptive batching 和 quantile regression 都显著优于 no batching。对 Scikit-Learn linear SVM, batching 可带来最高约 26x throughput improvement, 同时满足 20ms latency SLO。

4.5 Delayed batching

在中等或 bursty load 下, queue 里可能暂时没有足够请求组成最大 batch。Clipper 借鉴 Nagle's algorithm 的直觉, 短暂等待更多请求到达:

```
dispatch now:
  low latency
  smaller batch
  lower throughput

delay briefly:
  slightly higher latency
  larger batch
  higher throughput
```

论文 Figure 5 显示:

- Spark SVM 小 batch 已经比较高效, delayed batching 没明显收益。
- Scikit-Learn SVM 固定开销高, 2ms batch delay 带来 3.3x throughput improvement, 且仍在 10-20ms 交互式 latency 目标内。

这给 LLM serving 的启发是: dynamic batching 不是越等越好。等待窗口要和 SLO 绑定。

4.6 Model container 和 replica scaling

每个 model container 独立运行在 Docker container 里。这样做的理由:

- framework 不稳定不会拖垮整个 serving stack。
- 模型参数在初始化时加载, container 运行时尽量 stateless。

- 可以复制 replica 到多机或 GPU。
- 每个 replica 可以独立 adaptive batching, 因为机器和网络差异会影响 latency profile。

Figure 6 的 scaling 实验:

- 4-node GPU cluster。
- 10Gbps 网络时, 从 1 个 GPU replica 的 19,500 qps 扩到 4 个 replica 的 77,000 qps, 约 3.95x。
- 1Gbps 网络时, 第二个远程机器就可能让网络成为瓶颈。

这和今天 LLM serving 仍然一样:

- 如果瓶颈是 GPU compute, replica scaling 有效。
- 如果瓶颈是 network/input transfer, 加 replica 不一定线性提升。

5. Model Selection Layer

5.1 Selection policy interface

论文 Listing 2 的接口:

```
SelectionPolicy<S, X, Y>:
  init() -> S
  select(state, query) -> List<ModelId>
  combine(state, query, predictions) -> (prediction, confidence)
  observe(state, query, feedback, predictions) -> S
```

这四个函数把 model selection 抽象成一个可替换策略:

- select: 选择哪些模型。
- combine: 合并预测。
- observe: 用反馈更新 state。
- state: 可以为每个 user/session/context 单独维护。

今天的 LLM router 也可以这样写:

```
select:
  choose small, medium, large, or thinking model

combine:
  pick answer, vote, rerank, or judge

observe:
  update success rate, latency, cost, user feedback

state:
  per tenant, per session, per task type
```

5.2 Exp3 single model selection

Clipper 把单模型选择写成 multi-armed bandit。每个 model 是一个 arm。系统每次选择一个 model, 得到反馈 loss 或 reward, 然后更新权重。

论文使用 Exp3。简化管理:

```
weight[model] starts at 1
probability[model] = weight[model] / sum(weights)
```

choose model according to probability
observe feedback
increase weight for good model or reduce influence of bad model

论文公式里对 selected model 的 weight 做指数更新。不要被符号困住, 重点是:

- 它不需要一次调用所有模型。
- 它能在 feedback noisy 时逐步探索和利用。
- 它能在模型退化时转向其他模型。

Figure 8 模拟了 failure:

- CIFAR 上训练 5 个 Caffe model。
- 运行 20K sequential queries。
- 5K queries 后故意 degrade 最好模型。
- 10K queries 后让它恢复。
- Exp3 和 Exp4 都能快速减轻坏模型影响, 后续又逐步回到恢复的模型。

这是 production serving 的关键: model quality 不是静态的。

5.3 Exp4 ensemble model selection

ensemble 可以提升准确率, 但需要调用多个模型。Clipper 用 Exp4 来学习模型组合权重。

论文 Figure 7:

- CIFAR-10 和 ImageNet 上用 5 个 computer vision model 做 ensemble。
- ImageNet 上 ensemble 相比 single model 有 5.2% relative reduction in error。
- 当 4 个或 5 个模型 agree 时, prediction confidence 更高, error 更低。

这说明 model selection layer 不只是省成本, 也可以提高 robustness 和 confidence。

5.4 Straggler mitigation

ensemble 越大, tail latency 越容易被 straggler 拖垮。Figure 9 展示 ensemble size 增大时:

- P99 latency 很快超过 20ms latency objective。
- missing predictions 增加。
- 如果 deadline 到了就用 available predictions, accuracy 会略降, 但 latency 被控制住。

Clipper 的原则非常生产化:

late prediction may be worse than slightly less accurate prediction.

做法:

```
for each query:
  maintain deadline from latency SLO
  at deadline:
    combine only predictions that arrived
    substitute missing predictions with defaults or average
    reduce confidence according to missing fraction
```

这和 LLM tool/agent serving 很像: 如果某个 tool 或 expensive model 超时, 你要么返回 fallback, 要么降级, 不能让用户无限等待。

5.5 Contextualization

Clipper 支持为 user, context, session 维护独立 selection state。论文用 TIMIT speech recognition 做例子:

- 不同 dialect 用户适合不同 speech model。
- Clipper 的 selection policy 能用 feedback 快速找到更合适的 model combination。
- Figure 10 显示 Clipper selection policy 可以超过用户自报 dialect model。

对 LLM router 来说, 这意味着:

- 不同 tenant 可能适合不同模型。
- 不同任务类型需要不同 routing。
- 用户反馈不应该只进训练, 也可以进 serving selection state。

6. 和 TensorFlow Serving 的对比

TensorFlow Serving 是当时 Google 的生产级 serving 系统, 和 TensorFlow 深度集成。Clipper 和它比较的重点不是“谁绝对更快”, 而是:

Clipper 的 modular, container-based, cross-framework design 是否会带来不可接受的性能损失?

实验:

- MNIST 4-layer CNN。
- CIFAR-10 AlexNet。
- ImageNet Inception-v3。
- TensorFlow Serving 使用手工调优 batch size。
- Clipper 分别用 TensorFlow Python API 和 C++ API container。

结果:

- Clipper C++ containers 几乎达到 TensorFlow Serving 同等 throughput。
- Clipper Python containers 有 15-18% throughput hit。
- 主要瓶颈在 GPU inference 和 queueing, RPC overhead 很小。
- 模块化架构没有在核心 prediction-serving task 上造成显著性能惩罚。

这很重要: Clipper 不是为了功能牺牲性能。它试图证明“多框架、多模型、可反馈、可组合”这些生产能力可以用很小开销换来。

7. 局限性

论文第 7 节很直接:

Clipper 把模型当黑盒, 所以:

- 它不优化模型内部执行。
- slow model remains slow。
- 它不能像 TensorFlow Serving 那样利用 TensorFlow 内部 compilation 或 GPU acceleration 细节。
- 它不负责训练或 retraining。
- 如果所有 base models 都过时或不准确, Clipper 只能组合坏模型, 不能凭空变好。

今天读这段尤其重要。LLM serving 里, router/cache/batching/metrics 也不能替代:

- 更好的模型。

- 更快的 kernel。
- 更合理的量化。
- 更好的数据和训练。

Serving system 能控制的是围绕模型的运行路径, 不是模型本身的全部能力。

8. 和本仓库代码怎么连起来

8.1 本专题已有生产 serving 面

关键文件:

- learning/production-serving/src/openai_api_server.py
- learning/production-serving/src/streaming_sse.py
- learning/production-serving/src/metrics_prometheus.py
- learning/production-serving/src/cost_calc.py
- learning/production-serving/src/trtllm_build.py
- learning/production-serving/src/triton_model_repo/llm/config.pbtxt
- learning/production-serving/src/clipper_original_minimal.py

它们对应 Clipper 的不同层:

Application API:

openai_api_server.py
streaming_sse.py

Metrics and SLO:

metrics_prometheus.py

Cost and routing:

cost_calc.py

Model runtime / container shape:

trtllm_build.py
triton_model_repo

Clipper mechanisms:

clipper_original_minimal.py

8.2 新增 Clipper toy

clipper_original_minimal.py 包含:

PredictionCache:

LRU cache keyed by (model_id, query)

AdaptiveBatcher:

AIMD batch-size controller under latency SLO

ModelContainer:

toy latency profile and prediction method

best_effort_ensemble:

deadline-based straggler mitigation

exp3_probabilities and exp3_update:
tiny model-selection update

代码片段:

```
from clipper_original_minimal import AdaptiveBatcher, ModelContainer

batcher = AdaptiveBatcher(slo_ms=20.0, max_batch_size=4)
batcher.observe(batch_size=4, latency_ms=12.0)
print(batcher.max_batch_size)

batcher.observe(batch_size=5, latency_ms=25.0)
print(batcher.max_batch_size)

models = [
    ModelContainer("fast", fixed_ms=2.0, per_item_ms=1.0, accuracy=0.8),
    ModelContainer("slow", fixed_ms=50.0, per_item_ms=5.0, accuracy=0.95),
]
```

这段 toy 的学习价值:

- 当 batch latency 低于 SLO, batcher 增大 batch。
- 当 batch latency 超过 SLO, batcher 回退。
- deadline ensemble 会丢掉 slow model, 但 confidence 降低。
- Exp3 update 会提高被 reward 的模型概率。

8.3 测试对应论文 claim

新增测试:

- test_prediction_cache_lru_hit_and_evict
- test_adaptive_batcher_increases_then_backs_off
- test_batcher_respects_queue_and_current_cap
- test_best_effort_ensemble_drops_straggler_at_deadline
- test_exp3_update_increases_probability_after_reward

它们不是复现论文所有数字, 而是把论文机制压成可跑的最小行为:

cache:
repeated query can avoid recomputation

batching:
SLO controls batch size

straggler:
deadline controls how many predictions are used

selection:
feedback changes model probability

9. 和 LLM production serving 的迁移

Clipper 到 LLM 的迁移不要机械照搬。应该迁移的是系统原则。

9.1 API layer

Clipper 的 REST/RPC API 对应今天的 OpenAI-compatible endpoint:

```
/v1/models  
/v1/chat/completions  
stream=true SSE  
error schema  
usage token counts
```

本仓库 `openai_api_server.py` 就是在练这个 surface。生产里这层要稳定, 因为下游应用依赖它。

9.2 Batching and streaming

Clipper batching 是 prediction batch。LLM 里 batching 更复杂:

- prefill batching.
- decode continuous batching.
- streaming token 返回.
- KV cache 管理.
- speculative decoding.

但共同点是:

```
batching improves throughput,  
but waiting for batch hurts latency.  
SLO decides the acceptable tradeoff.
```

9.3 Cache

Clipper prediction cache:

```
(model_id, query) -> prediction
```

LLM cache 可能是:

```
prompt prefix -> KV cache  
prompt -> response  
embedding text -> vector  
retrieval query -> documents
```

每种 cache 都要回答:

- key 是什么.
- stale 怎么处理.
- 是否影响 safety/correctness.
- 命中率怎么监控.
- 节省多少 latency 和 cost.

9.4 Selection policy

Clipper 的 Exp3/Exp4 是传统 bandit。LLM router 可以从它学到:

- 不要只依赖离线 benchmark。
- 线上 feedback 可以更新 model choice。
- 每个 tenant/session 可以有不同 state。
- selection policy 必须同时看 accuracy, latency, cost。
- ensemble/judge 会提高质量, 但会增加 straggler 风险。

10. AI agent 学习方式

不要让 agent 只总结 Clipper。你要让它逼你把系统画出来。

Prompt:

我正在学 Clipper prediction serving。

请你一次只问一个问题, 按这个顺序考我:

1. training 和 inference 的系统挑战为什么不同
2. model abstraction layer 的职责
3. model selection layer 的职责
4. prediction cache 的 key 为什么必须包含 model_id
5. AIMD adaptive batching 如何用 SLO 控制 batch size
6. Exp3 和 Exp4 分别解决什么 serving 问题
7. straggler mitigation 为什么会用 accuracy 换 latency
8. 本仓库 clipper_original_minimal.py 哪个测试对应论文哪个机制

每次我回答后, 请指出一个漏洞, 并要求我用具体代码函数补充。

Agent 的正确角色是 examiner, reviewer, experiment coach。不是让它替你背论文。

11. 闭卷自测

读完你应该能回答:

- Clipper 为什么不是一个普通 RPC wrapper。
- 为什么 model abstraction layer 和 model selection layer 要分开。
- prediction cache 在 feedback join 中有什么作用。
- adaptive batching 为什么要以 latency SLO 为约束。
- AIMD 比 quantile regression 的工程优势是什么。
- delayed batching 和 Nagle's algorithm 的直觉相似在哪里。
- 为什么 model container 要进 Docker。
- Exp3 和 Exp4 的 serving tradeoff 是什么。
- straggler mitigation 为什么把 late prediction 变成 confidence loss。
- TensorFlow Serving 对比实验到底证明了什么, 没证明什么。
- Clipper 的局限为什么来自“模型在 Clipper 下方且是黑盒”。
- 本仓库 clipper_original_minimal.py 里的哪个类对应 prediction cache。
- 本仓库 metrics_prometheus.py 里的 histogram 如何承接 latency SLO。

12. 一句话复述模板

Clipper 解决的是在线 prediction serving 中, 模型部署、延迟、吞吐和准确率同时难以管理的问题。

它用 model abstraction layer
把应用和异构 ML framework 解耦。

它用 cache, adaptive batching 和 model containers
控制 latency/throughput。

它用 model selection layer,
通过 feedback, bandit, ensemble 和 straggler mitigation
提升准确率和鲁棒性。

实验显示它能在低延迟 SLO 下
获得大幅 batching 吞吐收益,
支持多模型和多框架,
并在 TensorFlow Serving 对比中保持相近性能。

它的局限是不能优化黑盒模型内部执行,
也不负责训练或模型过时问题。

真正掌握的标志: 你能把这个模板改写成仓库的 OpenAI API, SSE, metrics, cost 和 Clipper toy 代码
路径。